INSTITUTO TECNOLÓGICO DE COSTA RICA

Escuela de Ingeniería en Computadores

Tarea Nº 2

Documentación Técnica del Proyecto CE2048

Curso: Paradigmas de Programación (CE1106)

Semestre: Primer Semestre 2026

Código del proyecto: CE2048

Estudiantes:

Jorge Alberto Marín Navarro

Jimena Maria Castillo Campos

Justin Josué Solano Calderón

Profesor:

Msc. Ing. Luis Alonso Barboza Artavia

Fecha de entrega: 9 de abril de 2026

Índice

1. Descripción detallada del algoritmo de solución desarrollado y su justificación de uso **4**

1.1. Algoritmo general del sistema 4

1.2. Algoritmo de creación del tablero inicial 5

1.3. Algoritmo de movimiento y combinación 5

1.4. Algoritmo de inserción de una nueva baldosa 5

1.5. Algoritmo de detección de victoria y fin del juego 6

1.6. Justificación de la solución 6

1.7. Diagrama general del flujo del sistema 7

1.8. Diagrama de flujo del backend 8

1.9. Diagrama de flujo del frontend 9

2. Descripción de los procedimientos y funciones implementadas **9**

2.1. Funciones implementadas en `backend.rkt` 9

2.1.1. crear-tablero 9

2.1.2. mover-izquierda 9

2.1.3. mover-derecha 10

2.1.4. mover-arriba 10

2.1.5. mover-abajo 10

2.1.6. agregar-nueva-baldosa 10

2.1.7. hay-2048? 10

2.1.8. hay-movimientos? 10

2.1.9. tablero-igual? 10

2.1.10. filas-tablero y columnas-tablero 10

2.2. Funciones implementadas en `frontend.rkt` 11

2.2.1. color-celda 11

2.2.2. tamano-celda 11

2.2.3. crear-juego 11

2.2.4. iniciar-desde-campos 11
2.2.5. manejar-tecla 11
2.2.6. procesar-movimiento 11
2.2.7. aplicar-jugada 11
2.2.8. revisar-estado-juego 12
2.2.9. dibujar-escena 12
2.2.10. dibujar-tablero 12
2.2.11. dibujar-celda 12
2.2.12. actualizar-etiquetas 12

3. Descripción de la ejemplificación de las estructuras de datos desarrolladas 12

3.1. Representación lógica del tablero 12
3.2. Ejemplo de tablero 2x2 13
3.3. Ejemplo de tablero 4x4 13
3.4. Representación de un resultado de movimiento 13
3.5. Estructuras auxiliares 13

4. Problemas encontrados 14

4.1. Representación funcional del tablero 14
4.2. Reutilización de la lógica de movimientos 14
4.3. Detección correcta de movimientos válidos 14
4.4. Interfaz gráfica adaptable a tableros MxN 14
4.5. Integración entre backend y frontend 15

5. Problemas conocidos sin solución encontrados 15

5.1. Escalado visual en tableros muy grandes 15
5.2. Ajuste fino de centrado tipográfico 15
5.3. Probabilidad avanzada de inserción de baldosas 15

6. Plan de actividades realizadas por estudiante 15

7. Conclusiones y recomendaciones del proyecto **16**

8. Bitácora y repositorio **17**

9. Bibliografía consultada en todo el proyecto **17**

1. Descripción detallada del algoritmo de solución desarrollado y su justificación de uso

El proyecto CE2048 corresponde a una implementación del juego 2048 desarrollada en lenguaje Racket, orientada a reforzar conceptos de programación funcional, recursividad y manipulación de listas como estructuras de datos. El sistema fue construido en dos archivos principales: uno destinado a la lógica del juego y otro enfocado en la interfaz gráfica, en concordancia con los requerimientos no funcionales del proyecto. Asimismo, la solución implementa un tablero de tamaño $M \times N$, genera dos baldosas iniciales con valor 2 en posiciones aleatorias, permite el movimiento en las cuatro direcciones, acumula puntaje cuando ocurren fusiones válidas y detecta tanto la condición de victoria como el fin del juego. [1]

1.1. Algoritmo general del sistema

El sistema inicia cargando la interfaz gráfica y creando un tablero inicial con dimensiones configurables por el usuario. Una vez generado el tablero, el programa entra en un ciclo de interacción basado en eventos de teclado. Cuando el usuario presiona una flecha, el sistema ejecuta la lógica correspondiente al movimiento solicitado. Si el tablero cambia luego del desplazamiento, se actualiza el puntaje, se inserta una nueva baldosa aleatoria y se verifica si se alcanzó una baldosa con valor 2048 o si ya no existen movimientos posibles.

La estrategia general del sistema se puede resumir en los siguientes pasos:

1. Crear el tablero vacío de tamaño $M \times N$.
2. Insertar dos baldosas iniciales con valor 2 en posiciones distintas.
3. Mostrar el tablero en la interfaz gráfica.
4. Esperar una entrada del usuario mediante flechas del teclado.
5. Procesar el movimiento solicitado.
6. Calcular la nueva configuración del tablero y el puntaje asociado.
7. Si hubo cambio, insertar una nueva baldosa aleatoria.
8. Verificar condición de victoria o fin del juego.
9. Actualizar la interfaz y continuar la interacción.

1.2. Algoritmo de creación del tablero inicial

La creación del tablero parte de una representación matricial basada en listas. Primero se construye una matriz llena de ceros, donde cada cero representa una celda vacía. Posteriormente, se calcula la cantidad total de posiciones disponibles y se seleccionan dos índices distintos mediante generación aleatoria. Cada uno de esos índices se transforma en coordenadas de fila y columna, permitiendo colocar una baldosa con valor 2 en ambas posiciones.

Este algoritmo permite generalizar el juego a dimensiones distintas del tablero clásico 4x4, de acuerdo con el requerimiento funcional de soportar un tablero de tamaño $M \times N$. [1]

1.3. Algoritmo de movimiento y combinación

Cada movimiento se resuelve mediante una transformación funcional del tablero. Para el caso del desplazamiento hacia la izquierda, el procedimiento sobre cada fila sigue la secuencia:

1. Eliminar ceros de la fila.
2. Comparar elementos contiguos.
3. Fusionar únicamente pares iguales adyacentes.
4. Sumar el valor obtenido al puntaje acumulado.
5. Rellenar con ceros al final hasta recuperar el tamaño original.

Los demás movimientos se implementan a partir de transformaciones auxiliares sobre la matriz. El desplazamiento hacia la derecha se obtiene invirtiendo las filas, aplicando el algoritmo de mover a la izquierda y reconstruyendo la orientación original. Los movimientos verticales se resuelven mediante la transposición de la matriz y la reutilización de los algoritmos horizontales. Esta estrategia reduce duplicidad de lógica y mejora la consistencia del comportamiento del juego.

1.4. Algoritmo de inserción de una nueva baldosa

Una vez que un movimiento produce una modificación real del tablero, el sistema identifica todas las posiciones vacías. Luego selecciona una de ellas de manera aleatoria y coloca en esa casilla un 2 o un 4, con una selección aleatoria simple. Este procedimiento se ejecuta únicamente cuando el movimiento del usuario es válido, evitando que se agreguen nuevas baldosas si el tablero permanece igual.

1.5. Algoritmo de detección de victoria y fin del juego

La condición de victoria se verifica recorriendo todas las filas del tablero y comprobando si existe una baldosa con valor 2048. En caso afirmativo, el juego pasa al estado de victoria y la interfaz notifica al usuario.

La condición de continuidad del juego se determina mediante dos criterios:

1. Si existe al menos una celda vacía, entonces todavía hay movimientos posibles.
2. Si no hay celdas vacías, el sistema compara el tablero actual con el resultado de aplicar los cuatro movimientos posibles. Si ninguno de ellos produce un cambio, el juego concluye.

1.6. Justificación de la solución

La solución implementada resulta adecuada porque aprovecha de forma directa los principios del paradigma funcional exigidos en el proyecto. En particular, se modela el tablero como una estructura basada en listas, se favorece la recursividad para recorrer filas y matrices, y se separan claramente la lógica del juego y la interfaz gráfica. Además, la estrategia de reutilizar el movimiento hacia la izquierda como base para los otros tres desplazamientos simplifica la solución y reduce errores de mantenimiento.

Desde el punto de vista académico, este enfoque permite demostrar dominio de listas, procesamiento recursivo, transformaciones de estructuras matriciales y control de flujo en un entorno de interfaz gráfica, todo ello alineado con los objetivos del enunciado. [1]

1.7. Diagrama general del flujo del sistema

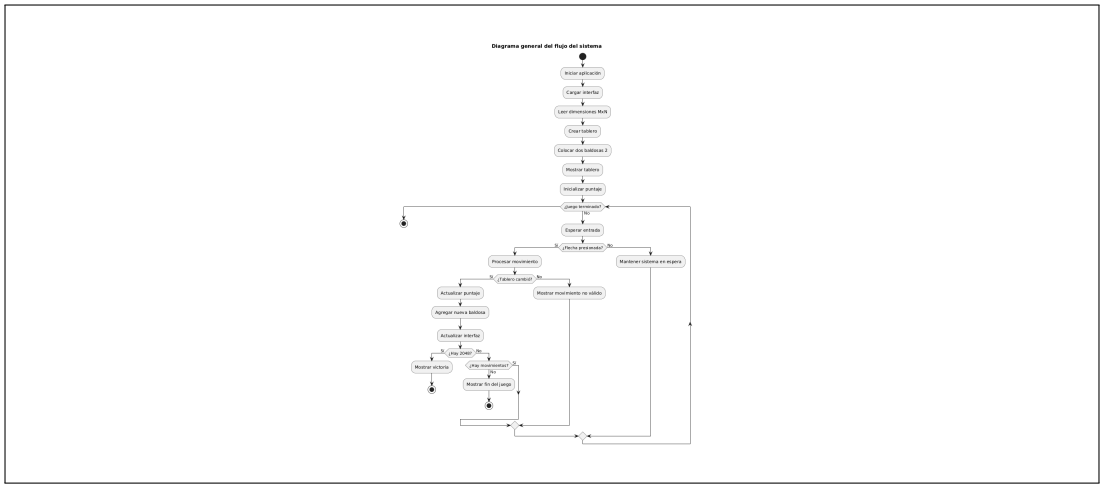

Figura 1: Diagrama general del flujo del sistema

1.8. Diagrama de flujo del backend

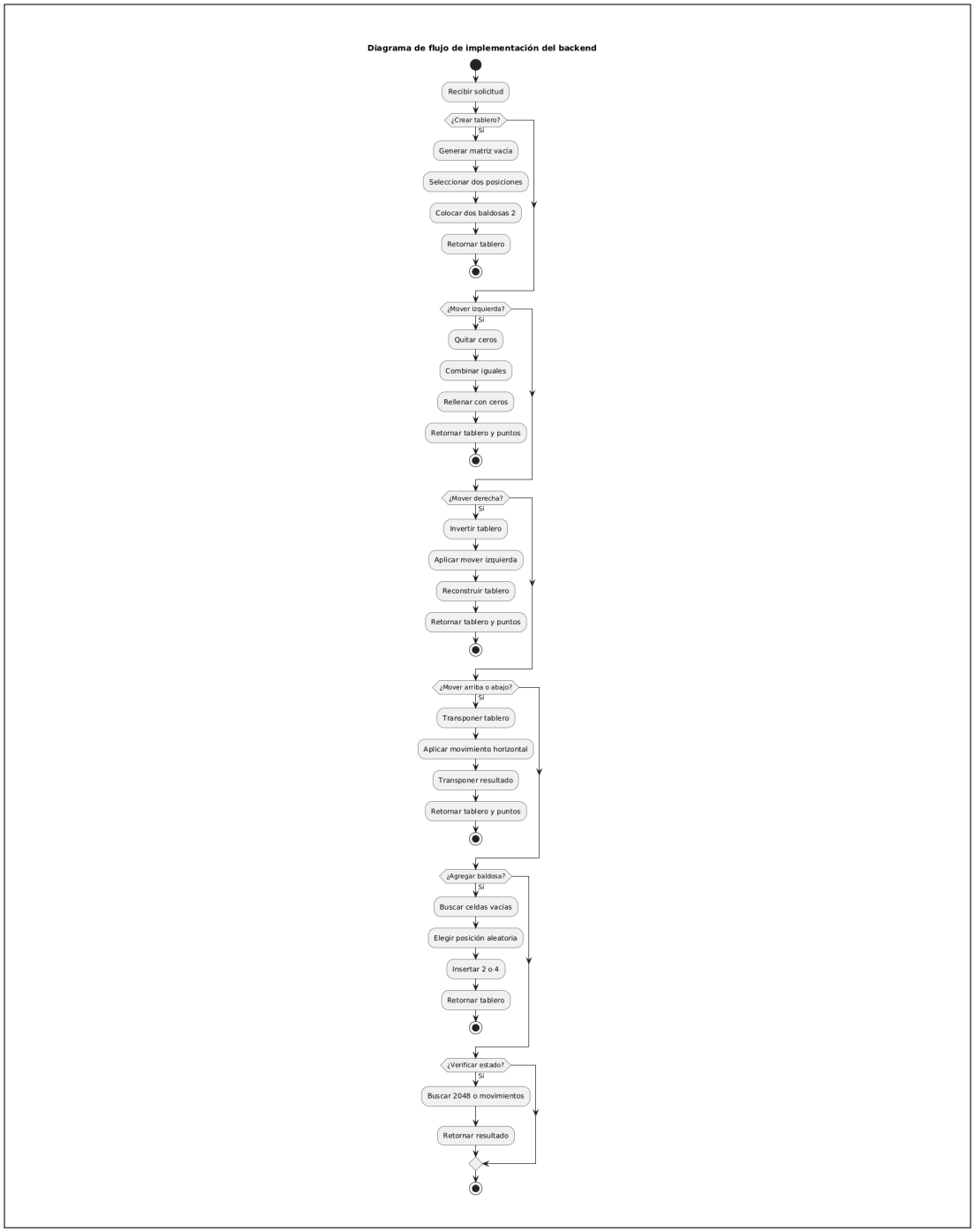

Figura 2: Diagrama de flujo de implementación del backend

1.9. Diagrama de flujo del frontend

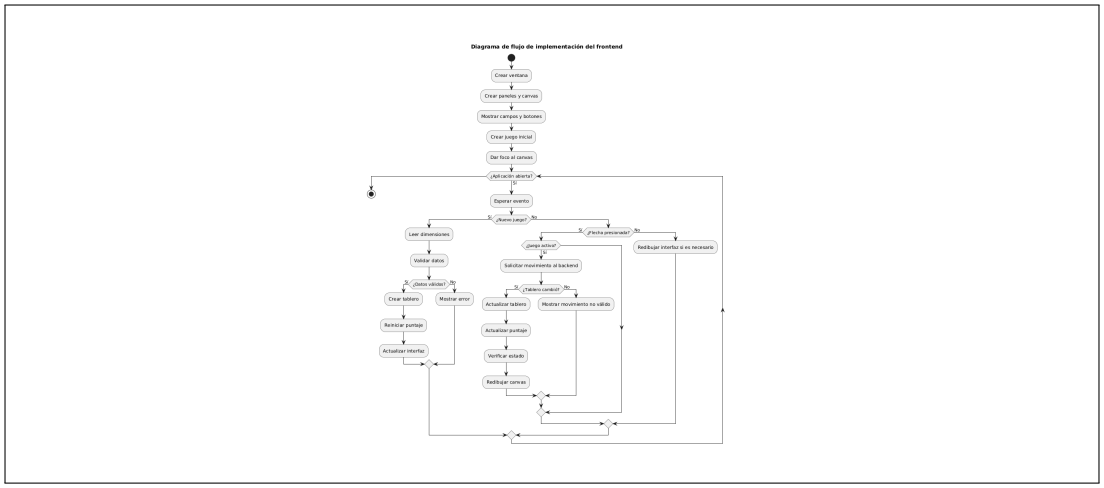

Figura 3: Diagrama de flujo de implementación del frontend

2. Descripción de los procedimientos y funciones implementadas

En esta sección se describen las funciones implementadas en los archivos `backend.rkt` y `frontend.rkt`. La separación entre ambos archivos responde al requerimiento del proyecto de utilizar al menos un archivo para la lógica del juego y otro para la interfaz gráfica. [1]

2.1. Funciones implementadas en `backend.rkt`

2.1.1. crear-tablero

Construye un tablero de tamaño $M \times N$. Internamente genera primero una matriz vacía de ceros y posteriormente coloca dos baldosas iniciales con valor 2 en posiciones aleatorias distintas.

2.1.2. mover-izquierda

Procesa el desplazamiento completo del tablero hacia la izquierda. Para cada fila elimina ceros, combina baldosas iguales una única vez por jugada, rellena con ceros y retorna tanto el nuevo tablero como los puntos obtenidos.

2.1.3. mover-derecha

Resuelve el desplazamiento hacia la derecha reutilizando el algoritmo de mover a la izquierda sobre una versión invertida del tablero.

2.1.4. mover-arriba

Resuelve el desplazamiento vertical superior mediante transposición del tablero, aplicación del algoritmo horizontal y reconstrucción de la orientación original.

2.1.5. mover-abajo

Implementa el desplazamiento vertical inferior mediante transposición y reutilización del movimiento hacia la derecha.

2.1.6. agregar-nueva-baldosa

Localiza las posiciones vacías del tablero y coloca aleatoriamente una nueva baldosa con valor 2 o 4.

2.1.7. hay-2048?

Recorre el tablero para detectar si existe una baldosa con valor 2048.

2.1.8. hay-movimientos?

Determina si el juego puede continuar. Verifica primero la existencia de celdas vacías y, si no existen, compara el tablero actual con los resultados de los cuatro movimientos posibles.

2.1.9. tablero-igual?

Compara dos tableros para determinar si un movimiento produjo o no un cambio real.

2.1.10. filas-tablero y columnas-tablero

Funciones auxiliares utilizadas para obtener las dimensiones actuales del tablero y apoyar tanto la lógica interna como el cálculo visual del frontend.

2.2. Funciones implementadas en `frontend.rkt`

2.2.1. color-celda

Asigna un color específico a cada valor posible de baldosa, con el fin de diferenciar visualmente el estado del tablero.

2.2.2. tamano-celda

Calcula dinámicamente el tamaño de las celdas que se deben dibujar en el canvas, de acuerdo con las dimensiones del tablero activo.

2.2.3. crear-juego

Inicializa el estado global de la partida, crea un tablero nuevo, reinicia el puntaje y actualiza la interfaz.

2.2.4. iniciar-desde-campos

Lee las dimensiones ingresadas por el usuario en la interfaz, las valida y, si son correctas, inicia una nueva partida.

2.2.5. manejar-tecla

Procesa los eventos de flechas del teclado y llama a la función de movimiento correspondiente.

2.2.6. procesar-movimiento

Verifica si el movimiento solicitado produce un cambio real. Si no existe cambio, informa al usuario mediante la interfaz; si sí existe cambio, delega la actualización completa del juego.

2.2.7. aplicar-jugada

Actualiza el tablero con el movimiento realizado, incrementa el puntaje y ejecuta la inserción de una nueva baldosa.

2.2.8. revisar-estado-juego

Comprueba si el usuario ganó la partida o si ya no existen movimientos disponibles.

2.2.9. dibujar-escena

Se encarga de pintar el lienzo principal del tablero y controlar la presentación visual general del área de juego.

2.2.10. dibujar-tablero

Recorre la matriz del tablero y delega el pintado de cada celda.

2.2.11. dibujar-celda

Representa visualmente una casilla individual del tablero, incluyendo borde, color de fondo y valor numérico si corresponde.

2.2.12. actualizar-etiquetas

Refresca los indicadores de puntaje, dimensión y estado del juego dentro de la interfaz.

3. Descripción de la ejemplificación de las estructuras de datos desarrolladas

La estructura central del sistema es una matriz representada mediante una lista de listas. Cada sublista corresponde a una fila del tablero y cada elemento numérico dentro de la sublista representa el valor almacenado en una casilla específica.

3.1. Representación lógica del tablero

El tablero se representa de manera conceptual como:

```
'((fila_1) (fila_2) ...(fila_m))
```

donde cada fila contiene N enteros y cada entero representa el valor de una baldosa.

3.2. Ejemplo de tablero 2x2

```
'((2 0)
  (0 2))
```

Esta estructura representa un tablero de 2x2. La casilla superior izquierda contiene un valor 2, la casilla inferior derecha contiene otro valor 2 y las dos posiciones restantes están vacías, representadas por 0. Esta forma de modelado coincide con el tipo de ejemplificación solicitado por el enunciado del proyecto. [1]

3.3. Ejemplo de tablero 4x4

```
'((2 0 0 2)
  (0 4 0 0)
  (2 2 8 0)
  (0 0 0 16))
```

En este caso, cada sublista modela una fila del tablero. La estructura permite aplicar transformaciones recursivas tanto por fila como por columna a través de procesos auxiliares como la transposición.

3.4. Representación de un resultado de movimiento

Los movimientos horizontales y verticales devuelven una estructura compuesta por dos partes:

```
(cons tablero-nuevo puntos-obtenidos)
```

Por tanto, el resultado de un movimiento combina:

- el nuevo tablero luego de procesar la jugada;
- la cantidad de puntos obtenidos por combinaciones válidas.

3.5. Estructuras auxiliares

Adicionalmente, el sistema utiliza estructuras auxiliares para apoyar la lógica funcional del juego:

- listas de ceros para construir filas iniciales;

- listas sin ceros para realizar fusiones;
- listas de posiciones vacías para insertar nuevas baldosas;
- matrices transpuestas e invertidas para reutilizar algoritmos de movimiento.

4. Problemas encontrados

Durante el desarrollo del proyecto se identificaron varios retos técnicos y de diseño.

4.1. Representación funcional del tablero

Uno de los principales retos fue modelar un tablero bidimensional sin recurrir a estructuras imperativas tradicionales. La solución adoptada consistió en utilizar listas de listas, lo cual permitió trabajar con un enfoque funcional y recursivo coherente con los objetivos del curso.

4.2. Reutilización de la lógica de movimientos

Implementar cuatro movimientos distintos podía provocar duplicación innecesaria de código. Como solución, se definió el movimiento hacia la izquierda como base del algoritmo y se construyeron los demás movimientos mediante inversión y transposición del tablero.

4.3. Detección correcta de movimientos válidos

Otro problema importante consistió en distinguir entre una tecla presionada y una jugada válida. La solución encontrada fue comparar el tablero anterior con el tablero resultante del movimiento. Solo si ambos son diferentes se agrega una nueva baldosa y se actualiza el estado del juego.

4.4. Interfaz gráfica adaptable a tableros MxN

La interfaz debía ser capaz de mostrar tableros de distintos tamaños sin perder legibilidad. Para ello se implementó un cálculo dinámico del tamaño de celda, ajustando la representación visual dentro del canvas según la cantidad de filas y columnas seleccionadas por el usuario.

4.5. Integración entre backend y frontend

Fue necesario mantener una separación real entre la lógica del juego y la presentación gráfica. La solución fue encapsular en el backend las funciones puramente funcionales del tablero y dejar en el frontend únicamente la gestión de eventos, dibujo y actualización visual.

5. Problemas conocidos sin solución encontrados

5.1. Escalado visual en tableros muy grandes

Aunque la interfaz admite tamaños variables, tableros excesivamente grandes pueden reducir demasiado el tamaño visible de cada celda, afectando la legibilidad de los valores.

5.2. Ajuste fino de centrado tipográfico

Dependiendo del valor numérico de la baldosa y del sistema operativo en el que se ejecute la aplicación, el centrado del texto dentro de las celdas puede requerir ajustes adicionales.

5.3. Probabilidad avanzada de inserción de baldosas

La implementación actual utiliza una selección simple entre 2 y 4. Una mejora futura sería ajustar las probabilidades de aparición para asemejar más el comportamiento clásico del juego.

6. Plan de actividades realizadas por estudiante

La siguiente tabla resume una propuesta de distribución del trabajo del proyecto. Debe ser ajustada con los nombres reales del equipo y las fechas efectivamente ejecutadas.

Actividad	**Tiempo estimado**	**Responsable**	**Estado**
Análisis del enunciado y definición de arquitectura base	2 horas	Jimena María Castillo Campos	Completado
Diseño de la representación del tablero y operaciones con listas	3 horas	Jimena María Castillo Campos	Completado

Actividad	Tiempo estimado	Responsable	Estado
Implementación de creación del tablero MxN	2 horas	Jimena María Castillo Campos	Completado
Implementación del movimiento hacia la izquierda	3 horas	Jimena María Castillo Campos	Completado
Implementación de inversión y transposición para reutilización de lógica	3 horas	Justin Josué Solano Calderón	Completado
Implementación de movimientos derecha, arriba y abajo	3 horas	Justin Josué Solano Calderón	Completado
Implementación de inserción aleatoria de nuevas baldosas	2 horas	Justin Josué Solano Calderón	Completado
Implementación de verificación de victoria y movimientos posibles	2 horas	Justin Josué Solano Calderón	Completado
Diseño de interfaz gráfica en Racket GUI	4 horas	Jorge Alberto Marín Navarro	Completado
Integración de eventos por flechas del teclado	2 horas	Jorge Alberto Marín Navarro	Completado
Pruebas de integración backend–frontend	3 horas	Todo el equipo	Completado
Corrección de errores y validación final	3 horas	Todo el equipo	Completado
Elaboración de documentación técnica en LaTeX	4 horas	Todo el equipo	Completadoo

7. Conclusiones y recomendaciones del proyecto

Se concluye que la implementación del proyecto CE2048 permitió aplicar de manera integrada conceptos fundamentales de programación funcional, recursividad y manipulación de listas. La representación del tablero como lista de listas resultó adecuada para expresar el problema con claridad y para construir transformaciones funcionales sobre la estructura del juego.

Asimismo, la separación entre backend y frontend permitió distribuir responsabilidades de forma ordenada: el archivo lógico concentra la manipulación del tablero y las reglas del juego, mientras que la interfaz gráfica administra la interacción del usuario y la representación

visual.

8. Bitácora y repositorio

La especificación del proyecto indica que la bitácora debe manejarse mediante un repositorio GitHub o GitLab, con avances incrementales reflejados en los commits y participación verificable de todos los integrantes. [1]

Enlace del repositorio:

https://github.com/JorgeMN2808IT/CE2048

9. Bibliografía consultada en todo el proyecto

Referencias

[1] Instituto Tecnológico de Costa Rica, Escuela de Ingeniería en Computadores. *Tarea #2: CE2048, Paradigmas de Programación, Primer Semestre 2026*. Documento del curso.

[2] Guzmán, J. E. *Introducción a la programación con Scheme*. Cartago: Editorial Tecnológica de Costa Rica, 2006.

[3] Racket. *The Racket Graphical Interface Toolkit*. Disponible en: http://download.racket-lang.org/releases/6.9/doc/gui/

[4] Wikipedia. *2048 (videojuego)*. Consulta de referencia conceptual para la mecánica general del juego.

[5] Material oficial del curso Paradigmas de Programación (CE1106), Instituto Tecnológico de Costa Rica.